DATA STRUCTURE IMPLEMENTATION (C\C++)
.SINGLY LInklist

```
#include <iostream>
using namespace std;

class Node {
public:
	int data;
	Node* next;

	Node(int value) {
		data = value;
		next = NULL;
	}
};

class LinkedList {
private:
	Node* head;

public:
	LinkedList() {
		head = NULL;
	}

	// Insert at end
	// Time Complexity: O(n)
	void insert(int value) {
		Node* newNode = new Node(value);

		if (head == NULL) {
			head = newNode;
			return;
		}

		Node* temp = head;
		while (temp->next != NULL) {
			temp = temp->next;
		}

		temp->next = newNode;
	}

	// Delete node
	// Time Complexity: O(n)
	void deleteNode(int value) {
		if (head == NULL) return;

		if (head->data == value) {
			Node* del = head;
			head = head->next;
			delete del;
			return;
		}
```

```
        Node* temp = head;

        while (temp->next != NULL && temp->next->data != value) {
            temp = temp->next;
        }

        if (temp->next == NULL) return;

        Node* del = temp->next;
        temp->next = temp->next->next;
        delete del;
    }

    // Search node
    // Time Complexity: O(n)
    bool search(int value) {
        Node* temp = head;

        while (temp != NULL) {
            if (temp->data == value)
                return true;

            temp = temp->next;
        }

        return false;
    }

    void display() {
        Node* temp = head;

        while (temp != NULL) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }

        cout << "NULL" << endl;
    }
};

int main() {
    LinkedList list;

    list.insert(10);
    list.insert(20);
    list.insert(30);

    cout << "Linked List: ";
    list.display();

    cout << "Search 20: ";
    cout << (list.search(20) ? "Found" : "Not Found") << endl;

    list.deleteNode(20);

    cout << "After deleting 20: ";
    list.display();
```

```
    return 0;
}
```

.STACK IMPLEMENTATION

```
#include <iostream>
using namespace std;

#define SIZE 100

class Stack {
private:
    int arr[SIZE];
    int top;

public:
    Stack() {
        top = -1;
    }

    // Push operation
    // Time Complexity: O(1)
    void push(int value) {
        if (top == SIZE - 1) {
            cout << "Stack Overflow\n";
            return;
        }

        arr[++top] = value;
    }

    // Pop operation
    // Time Complexity: O(1)
    void pop() {
        if (top == -1) {
            cout << "Stack Underflow\n";
            return;
        }

        top--;
    }

    // Search operation
    // Time Complexity: O(n)
    bool search(int value) {
        for (int i = 0; i <= top; i++) {
            if (arr[i] == value)
                return true;
        }

        return false;
    }

    void display() {
        for (int i = top; i >= 0; i--) {
```

```
                cout << arr[i] << endl;
            }
        }
};

int main() {
        Stack s;

        s.push(5);
        s.push(10);
        s.push(15);

        cout << "Stack Elements:\n";
        s.display();

        cout << "Search 10: ";
        cout << (s.search(10) ? "Found" : "Not Found") << endl;

        s.pop();

        cout << "After Pop:\n";
        s.display();

        return 0;
}
```

. Queue implementation

```
#include <iostream>
using namespace std;

#define SIZE 100

class Queue {
private:
        int arr[SIZE];
        int front, rear;

public:
        Queue() {
                front = 0;
                rear = -1;
        }

        // Enqueue operation
        // Time Complexity: O(1)
        void enqueue(int value) {
                if (rear == SIZE - 1) {
                        cout << "Queue Overflow\n";
                        return;
                }

                arr[++rear] = value;
        }

        // Dequeue operation
```

```
    // Time Complexity: O(1)
    void dequeue() {
        if (front > rear) {
            cout << "Queue Underflow\n";
            return;
        }

        front++;
    }

    // Search operation
    // Time Complexity: O(n)
    bool search(int value) {
        for (int i = front; i <= rear; i++) {
            if (arr[i] == value)
                return true;
        }

        return false;
    }

    void display() {
        for (int i = front; i <= rear; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};

int main() {
    Queue q;

    q.enqueue(1);
    q.enqueue(2);
    q.enqueue(3);

    cout << "Queue Elements: ";
    q.display();

    cout << "Search 2: ";
    cout << (q.search(2) ? "Found" : "Not Found") << endl;

    q.dequeue();

    cout << "After Dequeue: ";
    q.display();

    return 0;
}


.Binary search tree (BST)
#include <iostream>
using namespace std;

class Node {
public:
```

```
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = right = NULL;
    }
};

class BST {
private:
    Node* root;

    Node* insert(Node* node, int value) {
        if (node == NULL)
            return new Node(value);

        if (value < node->data)
            node->left = insert(node->left, value);
        else
            node->right = insert(node->right, value);

        return node;
    }

    bool search(Node* node, int value) {
        if (node == NULL)
            return false;

        if (node->data == value)
            return true;

        if (value < node->data)
            return search(node->left, value);

        return search(node->right, value);
    }

    Node* findMin(Node* node) {
        while (node->left != NULL)
            node = node->left;

        return node;
    }

    Node* deleteNode(Node* node, int value) {
        if (node == NULL)
            return NULL;

        if (value < node->data)
            node->left = deleteNode(node->left, value);

        else if (value > node->data)
            node->right = deleteNode(node->right, value);

        else {
```

```
            if (node->left == NULL) {
                Node* temp = node->right;
                delete node;
                return temp;
            }

            else if (node->right == NULL) {
                Node* temp = node->left;
                delete node;
                return temp;
            }

            Node* temp = findMin(node->right);

            node->data = temp->data;

            node->right = deleteNode(node->right, temp->data);
        }

        return node;
    }

    void inorder(Node* node) {
        if (node == NULL)
            return;

        inorder(node->left);
        cout << node->data << " ";
        inorder(node->right);
    }

public:
    BST() {
        root = NULL;
    }

    // Insert: O(log n) average
    void insert(int value) {
        root = insert(root, value);
    }

    // Search: O(log n) average
    bool search(int value) {
        return search(root, value);
    }

    // Delete: O(log n) average
    void deleteValue(int value) {
        root = deleteNode(root, value);
    }

    void display() {
        inorder(root);
        cout << endl;
    }
};
```

```
int main() {
    BST tree;

    tree.insert(50);
    tree.insert(30);
    tree.insert(70);
    tree.insert(20);
    tree.insert(40);

    cout << "BST Inorder Traversal: ";
    tree.display();

    cout << "Search 40: ";
    cout << (tree.search(40) ? "Found" : "Not Found") << endl;

    tree.deleteValue(30);

    cout << "After deleting 30: ";
    tree.display();

    return 0;
}
```

COMPLEXITY SUMMARY

Data Structure	Insert	Delete	Search
Linked List	O(n)	O(n)	O(n)
Stack	O(1)	O(1)	O(n)
Queue	O(1)	O(1)	O(n)
BST	O(log n)*	O(log n)*	O(log n)*

* IN CASE OF AVERAGE COMPLEXITY FOR BST . WORST CASE IS O(n)

Linked List:-

Sample Input & Output
Linked List: 10 -> 20 -> 30 -> NULL
Search 20: Found
After deleting 20:
10 -> 30 -> NULL

Short Explanation:
The program inserts three elements into the linked list.
It searches for element 20, which is found successfully.
After deleting 20, the updated linked list is displayed.

2. Stack:-

Sample Input & Output
Stack Elements:
15
10
5
Search 10: Found
After Pop:

10
5

Short Explanation:
The stack stores elements using the LIFO (Last In First Out) principle.
Elements 5, 10, and 15 are pushed into the stack.
After one pop operation, the top element (15) is removed.

3. Queue:-

Sample Input & Output
Queue Elements: 1 2 3
Search 2: Found
After Dequeue:
2 3

Short Explanation:
The queue follows the FIFO (First In First Out) principle.
Elements 1, 2, and 3 are inserted into the queue.
After one dequeue operation, the first element (1) is removed.

4. Binary Search Tree (BST):-

Sample Input & Output
BST In order Traversal: 20 30 40 50 70
Search 40: Found
After deleting 30:
20 40 50 70

Short Explanation:
The BST stores data in sorted hierarchical form.
Several nodes are inserted and displayed using in order traversal.
After deleting node 30, the tree is updated and displayed again.

Short Project Explanation

This project demonstrates the implementation of basic data structures in C++ without using STL containers. The goal is to understand how memory management, insertion, deletion, and searching operations work internally.

The project includes:

* Linked List for dynamic memory storage
* Stack for LIFO operations
* Queue for FIFO operations
* Binary Search Tree for hierarchical sorted storage

Each data structure is implemented using classes and tested using driver programs.